

Forogaming

Foro comunitario de Videojuegos — Guías, Easter Eggs & Reviews

Documento	Documentación técnica del proyecto
Versión	1.2.0
Fecha	10 de abril de 2026
Estado	Producción · https://forogaming.vercel.app
Stack	Next.js 16 · TypeScript · PostgreSQL (Neon) · Tailwind CSS
Servicios	Vercel · Neon · Uploadthing · IGDB API

DOCUMENTACIÓN GENERADA AUTOMÁTICAMENTE

Índice de contenidos

- | | | |
|----|--------------------------------|---|
| 1. | Visión general del proyecto | Propósito, audiencia y stack tecnológico |
| 2. | Instalación paso a paso | Entorno, dependencias, variables de entorno y migraciones |
| 3. | Estructura del proyecto | Árbol de carpetas y descripción de archivos clave |
| 4. | Base de datos | Tablas, campos, relaciones e índices |
| 5. | API y endpoints | Lista completa de rutas, métodos y respuestas |
| 6. | Sistema de autenticación | JWT, refresh tokens, proxy y flujo completo |
| 7. | Correcciones aplicadas | Deprecaciones resueltas en Next.js 16 |
| 8. | Estado actual y próximos pasos | Qué funciona, qué falta y roadmap |

¿Qué es Forogaming?

Forogaming es una plataforma web de tipo foro comunitario especializada en videojuegos. Permite a los usuarios publicar y descubrir **guías paso a paso**, **easter eggs** y **reviews** de títulos, con un sistema de interacción social completo: votos, likes, comentarios anidados y favoritos.

El estilo de interacción es similar a Reddit: feed principal con posts ordenables (nuevo, top, trending), sidebar con navegación por juego y categoría, y vista de artículo con multimedia y pasos numerados. El diseño es oscuro y gaming-first, responsive con prioridad desktop.

Público objetivo

- Jugadores que buscan guías detalladas y trucos para sus juegos favoritos.
- Creadores de contenido que quieren publicar análisis y descubrimientos.
- Comunidades de juegos específicos que necesitan un espacio propio de discusión.

Funcionalidades principales

Módulo	Funcionalidad	Estado
Posts	CRUD completo con categorías, multimedia, steps numerados y soft-delete	 Implementado
Comentarios	Árbol anidado con 2–4 niveles (adjacency list, ADR-002)	 Implementado
Votos	Upvote / Downvote en posts y comentarios, impacta el ranking	 Implementado
Likes	Interacción emocional independiente del ranking (ADR-005)	 Implementado
Favoritos	Guardar posts para lectura posterior	 Implementado
Trending	Score tipo Reddit: $(\text{votos_netos}) / (\text{horas} + 2)^{1.8}$ — vista SQL	 Implementado
Auth	JWT (15 min) + refresh token (7 días, httpOnly cookie)	 Implementado
Moderación	Reportes, baneos, eliminación de contenido por moderadores	 Implementado
Búsqueda	Full-text con ILIKE en título y cuerpo	 Implementado
Upload imágenes	Avatares (2 MB) e imágenes de posts (8 MB) vía Uploadthing	 Implementado
IGDB	Búsqueda de juegos en tiempo real, páginas enriquecidas con portada, capturas, géneros y rating	 Implementado
Seguir usuarios	Follow / unfollow con contador de seguidores/seguídos en el perfil	 Implementado
Mensajería directa	Chat privado con polling cada 3 s, marca de leído (✓), badge de no leídos en Navbar	 Implementado
Sistema de amigos	Solicitudes de amistad, aceptar/rechazar, lista de amigos, badge de solicitudes pendientes	 Implementado
Configuración de perfil	Página /settings: cambiar avatar (Uploadthing) y bio	 Implementado

Stack tecnológico completo

Capa	Tecnología	Versión	Justificación
Framework	Next.js (App Router)	16.2.3	SSR nativo para SEO de posts públicos, fullstack en un solo repo
Lenguaje	TypeScript	5.x (strict)	Tipado fuerte en todo el proyecto, sin <code>any</code>
Estilos	Tailwind CSS	3.4	Utility-first, tema oscuro gaming con variables CSS
Base de datos	PostgreSQL	17.9	Relacional, soporte ENUM nativo, extensión pgcrypto
Cliente DB	pg (node-postgres)	8.11	Pool de conexiones, queries tipadas, transacciones
Auth tokens	jose	5.x	JWT compatible con Edge Runtime de Next.js
Passwords	bcryptjs	2.4	Hashing con cost factor 12 (SDD \$5.4)
Validación	Zod	3.22	Schemas en servidor y cliente, formateo de errores unificado
Sanitización	Implementación nativa	—	Reemplaza isomorphic-dompurify (incompatible con Turbopack/ESM). Prevención XSS sin dependencias externas.
Rate limiting	Implementación propia	—	Sliding window en memoria; migrar a Redis en v2
Upload de imágenes	Uploadthing	v7	Avatares (2 MB) e imágenes de posts (8 MB). Auth vía JWT en el file router.
API de juegos	IGDB (Twitch)	v4	Búsqueda Apicalypse, token OAuth cached en módulo, cover CDN. Enriquece páginas de juego con screenshots, géneros y rating.
Runtime	Node.js	22.20	Versión LTS activa

Decisiones de diseño clave (ADRs)

- **ADR-001:** Stack Next.js fullstack (Opción A) — SSR, ecosistema maduro, menor overhead.
- **ADR-002:** Comentarios con adjacency list (`parent_id`) — simple, suficiente para 2–4 niveles.
- **ADR-003:** Trending con score tipo Reddit recalculado sobre vista SQL `post_scores` .
- **ADR-004:** Upload con Uploadthing v7 — file router con auth JWT, sin configuración de S3 propia. Avatares 2 MB, imágenes de posts 8 MB.
- **ADR-005:** Votos y likes son sistemas separados — votos afectan ranking, likes son emocionales.
- **ADR-006:** Mensajería con polling cada 3 s (no WebSocket) — compatible con Vercel serverless, suficiente para el volumen actual.
- **ADR-007:** IGDB como fuente de metadatos de juegos — Twitch OAuth con token cacheado en módulo, query Apicalypse sin cláusula `where` (incompatible con `search`).

- **ADR-008:** Sanitización HTML nativa sin dependencias — reemplaza isomorphic-dompurify que fallaba en Vercel/Turbopack por incompatibilidad ESM de jsdom → @exodus/bytes.

2 Instalación paso a paso

A continuación se documenta cada paso realizado para poner en marcha el entorno de desarrollo desde cero en Windows 11.

Requisitos previos del sistema

Software	Versión instalada	Verificación
Windows 11 Home	10.0.26200	Sistema operativo del entorno
Node.js	22.20.0 (LTS)	<code>node --version</code>
npm	10.9.3	<code>npm --version</code>
winget	v1.28.220	<code>winget --version</code>
PostgreSQL 17	17.9	Instalado en este proceso
Microsoft Edge	Chromium	Preinstalado en Windows 11

Pasos de instalación

1 Instalar PostgreSQL 17 con winget

Se verificó que PostgreSQL no estaba instalado (ni en PATH, ni en Program Files, ni como servicio Windows). Se usó el gestor de paquetes `winget` — disponible en Windows 11 de forma nativa — para instalar la versión oficial 17.9.

```
winget install --id PostgreSQL.PostgreSQL.17 --accept-package-agreements --accept-source-agreements
```

El instalador gráfico de EnterpriseDB descargó 354 MB e instaló PostgreSQL con contraseña `postgres` para el superusuario, puerto 5432 y servicio de Windows registrado como `postgresql-x64-17`.

✓ Resultado: PostgreSQL 17.9 instalado en `C:\Program Files (x86)\PostgreSQL\17\`

2

Verificar y arrancar el servicio PostgreSQL

Se comprobó el estado del servicio de Windows con `sc query`. El instalador lo registró y arrancó automáticamente.

```
sc query type= all state= all | grep postgres
# NOMBRE_SERVICIO: postgresql-x64-17
# ESTADO: 4    RUNNING
```

✓ Servicio activo en localhost:5432. Conexión verificada: `PGPASSWORD=postgres psql -U postgres -c "\l"`

3

Crear la base de datos del proyecto

Se creó la base de datos `forogaming` con codificación UTF8 usando el superusuario `postgres`.

```
PGPASSWORD=postgres psql -U postgres -c "CREATE DATABASE forogaming ENCODING 'UTF8';"
```

Por qué:

la aplicación necesita una base de datos dedicada, separada de la base `postgres` por defecto. Usar una base propia permite gestionar permisos, backups y migraciones de forma aislada.

✓ Base de datos `forogaming` creada correctamente.

4

Instalar dependencias npm

Desde el directorio del proyecto, se instalaron las 434 dependencias declaradas en `package.json`.

```
cd C:\Users\nicol\Desktop\Forogaming
npm install
```

Las dependencias principales instaladas incluyen: `next`, `react`, `pg`, `jose`, `bcryptjs`, `zod`, `isomorphic-dompurify`, `uuid`, `slugify`.

⚠ Se detectó CVE-2025-66478 en Next.js 15.1.0. Se actualizó inmediatamente a 16.2.3 (ver paso 6).

5

Configurar variables de entorno (.env.local)

Se copió el archivo de ejemplo y se rellenaron automáticamente los dos valores críticos:

```
# .env.local generado automáticamente
DATABASE_URL=postgresql://postgres:postgres@localhost:5432/forogaming
JWT_SECRET=uSxbN5HHIZvITzfkr7PWVNAf9DmcrfQ3u0ynXRMz7aIq7PYf
JWT_ACCESS_EXPIRES=15m
JWT_REFRESH_EXPIRES=7d
NEXT_PUBLIC_APP_URL=http://localhost:3000
APP_ENV=development
```

- **DATABASE_URL:**

cadena de conexión que usa el pool de `pg` para conectarse a PostgreSQL.

- **JWT_SECRET:**

secreto criptográficamente aleatorio de 48 caracteres (generado con

`crypto.randomBytes(36).toString('base64url')`) usado para firmar y verificar todos los JWT.

 `.env.local` está en `.gitignore` y nunca se sube al repositorio.

6

Ejecutar las migraciones de base de datos

El script `scripts/migrate.js` lee el archivo `migrations/001_initial_schema.sql`, se conecta a PostgreSQL usando `DATABASE_URL` de `.env.local` y ejecuta el SQL completo en una sola transacción.

```
npm run db:migrate
# [migrate] Cargado .env.local
# [migrate] Ejecutando migración 001_initial_schema.sql ...
# [migrate] ✅ Migración completada exitosamente
```

Efecto:

se crearon 7 tipos ENUM, 11 tablas, 10 índices, 3 triggers de `updated_at` y la vista `post_scores` para trending.

✅ 11 tablas verificadas con `dt` en `psql`.

7

Actualizar Next.js (parche de seguridad CVE-2025-66478)

La versión 15.1.0 declarada en `package.json` tenía una vulnerabilidad crítica. Se actualizó a la última versión estable antes de arrancar el servidor.

```
npm install next@latest
# Resultado: Next.js 16.2.3 instalado, 0 vulnerabilidades
```

```
npm run dev
# ▲ Next.js 16.2.3 (Turbopack)
# - Local: http://localhost:3000
# ✓ Ready in 408ms
```

Verificación del health check:

```
curl http://localhost:3000/api/health
{"status":"ok","db":"connected","timestamp":"2026-04-09T16:52:53.379Z"}
```

✓ Servidor corriendo. API responde. Base de datos conectada.

Comandos útiles de mantenimiento

Comando	Qué hace
<code>npm run dev</code>	Arranca el servidor de desarrollo con Turbopack (hot reload)
<code>npm run build</code>	Compila el proyecto para producción
<code>npm run start</code>	Arranca el servidor de producción (requiere build previo)
<code>npm run db:migrate</code>	Ejecuta las migraciones SQL pendientes
<code>npm run lint</code>	Ejecuta ESLint sobre todo el código fuente

3 Estructura del proyecto

Forogaming/ |— .env.example ← Variables de entorno plantilla (sin secretos) |— .env.local ← Variables reales (ignorado por git) |— .gitignore |— package.json ← Dependencias y scripts npm |— tsconfig.json ← TypeScript strict mode, alias @/* |— next.config.ts ← Security headers, remotePatterns |— tailwind.config.ts ← Tema oscuro gaming |— postcss.config.js |— SDD.md ← Software Design Document (origen del proyecto) |— documentacion-proyecto.pdf ← Este documento | |— migrations/ | |— 001_initial_schema.sql ← Schema completo de PostgreSQL | |— scripts/ | |— migrate.js ← Runner de migraciones SQL | |— generate-docs.js ← Generador de este PDF | |— src/ | |— proxy.ts ← Proxy global (antes middleware.ts) - inyecta headers de usuario | |— types/ | |— index.ts ← Interfaces TypeScript de todas las entidades del dominio | |— contexts/ | |— AuthContext.tsx ← Estado de autenticación global (React Context + hooks) | |— lib/ ← Utilidades compartidas del servidor | |— db.ts ← Pool de conexiones PostgreSQL, query(), withTransaction() | |— auth.ts ← JWT: signAccessToken, signRefreshToken, verifyAccessToken | |— password.ts ← hashPassword(), verifyPassword() con bcrypt cost 12 | |— ratelimit.ts ← Rate limiter sliding window en memoria | |— sanitize.ts ← Sanitización XSS nativa (sin dependencias externas) | |— validation.ts ← Schemas Zod para todos los inputs de API | |— uploadthing.ts ← generateReactHelpers para useUploadThing en el cliente | |— igdb.ts ← Twitch OAuth + IGDB Apicalypse: searchIGDBGames, getIGDBGGameDetails | |— components/ ← Componentes React reutilizables | |— Navbar.tsx ← Logo, búsqueda, menú usuario, badge mensajes y amigos | |— Sidebar.tsx ← Navegación lateral: categorías y juegos | |— Feed.tsx ← Lista de PostCards con paginación | |— PostCard.tsx ← Tarjeta de post con votos, badges y metadatos | |— VoteButtons.tsx ← Botones ▲/▼ con optimistic updates | |— CommentTree.tsx ← Árbol de comentarios anidados recursivo | |— CommentForm.tsx ← Formulario inline de nuevo comentario / respuesta | |— GameSearch.tsx ← Buscador IGDB con debounce para el formulario de post | |— FollowButton.tsx ← Botón seguir/dejar de seguir con estado real desde el cliente | |— AddFriendModal.tsx ← Modal de búsqueda de usuarios y envío de solicitud de amistad | |— app/ ← Next.js App Router | |— layout.tsx ← Root layout: AuthProvider + Navbar + main | |— page.tsx ← / - Home con feed principal y trending | |— globals.css ← Tailwind base + variables CSS tema oscuro | |— (auth)/ | |— login/page.tsx | |— register/page.tsx | |— game/[slug]/page.tsx ← Página enriquecida: hero IGDB, screenshots, rating, géneros | |— games/page.tsx ← Catálogo de juegos con portadas | |— post/[id]/page.tsx ← Vista completa del post + comentarios | |— category/[type]/page.tsx ← Feed por categoría | |— search/page.tsx ← Búsqueda full-text | |— user/[username]/page.tsx ← Perfil público con FollowButton, stats y posts | |— settings/page.tsx ← Ajustes de perfil: avatar (Uploadthing) y bio | |— messages/page.tsx ← Lista de conversaciones con última mensaje y no leídos | |— messages/[username]/page.tsx ← Chat con polling 3 s, separadores de fecha, ✓ de leído | |— friends/page.tsx ← Lista de amigos + solicitudes pendientes con aceptar/rechazar | |— submit/page.tsx ← Crear post con buscador IGDB integrado | |— admin/page.tsx ← Panel de moderación | |— admin/reports/page.tsx ← Gestión de reportes | |— admin/users/page.tsx ← Gestión de usuarios | |— api/ ← 40+ rutas API REST | |— auth/ register · login · refresh · logout | |— posts/ CRUD · trending · search · [id]/comments | |— comments/[id]/ | |— votes/ · likes/ · favorites/[postId]/ | |— games/ · games/search/ IGDB search + upsert en DB | |— users/ me · [username] · [username]/follow · search | |— messages/ conversaciones · [username] · unread | |— friends/ lista · request · respond · pending | |— uploadthing/ file router: avatarUploader · postImageUploader | |— reports/ · admin/ | |— health/

Motor: **PostgreSQL 17**. Base de datos: `forogaming`. La migración crea todo el schema desde cero con el comando `npm run db:migrate`.

Tipos ENUM personalizados

Tipo	Valores posibles
<code>user_role</code>	<code>admin</code> · <code>moderator</code> · <code>user</code> · <code>guest</code>
<code>post_category</code>	<code>guide</code> · <code>easter-egg</code> · <code>review</code> · <code>general</code>
<code>media_type</code>	<code>image</code> · <code>video_embed</code>
<code>vote_target</code>	<code>post</code> · <code>comment</code>
<code>like_target</code>	<code>post</code> · <code>comment</code>
<code>report_target</code>	<code>post</code> · <code>comment</code> · <code>user</code>
<code>report_status</code>	<code>pending</code> · <code>resolved</code> · <code>dismissed</code>

Tablas

users — Usuarios registrados

Campo	Tipo	Descripción
<code>id</code>	UUID PK	Generado con <code>gen_random_uuid()</code> (extensión pgcrypto)
<code>username</code>	VARCHAR(50) UNIQUE	Nombre de usuario público, solo <code>[a-zA-Z0-9_-]</code>
<code>email</code>	VARCHAR(255) UNIQUE	Email de acceso, nunca se devuelve en endpoints públicos
<code>password_hash</code>	VARCHAR(255)	Bcrypt con cost factor 12
<code>role</code>	user_role	Default <code>user</code>
<code>avatar_url</code>	TEXT nullable	URL HTTPS de avatar
<code>bio</code>	TEXT nullable	Descripción del perfil
<code>is_banned</code>	BOOLEAN	Default <code>false</code> ; si <code>true</code> , login rechazado con 403
<code>created_at</code>	TIMESTAMP TZ	Automático
<code>updated_at</code>	TIMESTAMP TZ	Actualizado por trigger automático

Campo	Tipo	Descripción
<code>id</code>	UUID PK	Auto-generado
<code>name</code>	VARCHAR(255)	Nombre visible del juego
<code>slug</code>	VARCHAR(255) UNIQUE	URL-friendly: <code>the-witcher-3</code>
<code>cover_url</code>	TEXT nullable	URL de la imagen de portada
<code>description</code>	TEXT nullable	Descripción del juego
<code>created_at</code>	TIMESTAMPTZ	Automático

posts — Publicaciones del foro

Campo	Tipo	Descripción
<code>id</code>	UUID PK	Auto-generado
<code>title</code>	VARCHAR(500)	Título del post
<code>body</code>	TEXT	Contenido HTML sanitizado con DOMPurify
<code>category</code>	post_category	Categoría del post
<code>game_id</code>	UUID FK → games	Juego asociado (nullable)
<code>author_id</code>	UUID FK → users	Autor (SET NULL si se borra el usuario)
<code>is_published</code>	BOOLEAN	Default <code>true</code> ; <code>false</code> = borrador
<code>is_deleted</code>	BOOLEAN	Soft-delete: el registro permanece pero no se muestra
<code>view_count</code>	INTEGER	Incrementado en cada GET (fire & forget)
<code>created_at / updated_at</code>	TIMESTAMPTZ	Trigger automático en UPDATE

Otras tablas del schema inicial

Tabla	Propósito	Claves foráneas
post_media	Imágenes y embeds de vídeo por post	post_id → posts (CASCADE)
post_steps	Pasos numerados para guías	post_id → posts (CASCADE); UNIQUE(post_id, step_num)
comments	Comentarios con threading (adjacency list)	post_id → posts; author_id → users; parent_id → comments
votes	Upvote (+1) / Downvote (-1) en posts y comentarios	user_id → users; UNIQUE(user_id, target_type, target_id)
likes	Likes emocionales (no afectan ranking)	user_id → users; UNIQUE(user_id, target_type, target_id)
favorites	Posts guardados por el usuario	user_id → users; post_id → posts; UNIQUE(user_id, post_id)
reports	Denuncias de contenido o usuarios	reporter_id → users
revoked_tokens	Lista negra de refresh tokens revocados en logout	jti (PK VARCHAR)

Tablas añadidas en producción (Neon)

Tabla	Campos principales	Propósito
follows	follower_id UUID FK → users following_id UUID FK → users UNIQUE(follower_id, following_id)	Sistema de seguimiento entre usuarios. Un row = un follow.
direct_messages	id UUID PK sender_id UUID FK → users receiver_id UUID FK → users body TEXT (max 2000) read_at TIMESTAMPTZ nullable created_at TIMESTAMPTZ	Mensajes directos entre usuarios. read_at NULL = no leído. El receptor los marca como leídos al abrir el chat.
friend_requests	id UUID PK sender_id UUID FK → users receiver_id UUID FK → users status TEXT CHECK (pending/accepted/rejected) created_at TIMESTAMPTZ UNIQUE(sender_id, receiver_id)	Solicitudes de amistad. Auto-acepta si ya existe solicitud inversa pendiente. La restricción UNIQUE evita duplicados.

Vista: post_scores (trending)

Vista SQL que calcula el score de trending en tiempo real usando el algoritmo tipo Reddit (ADR-003):

$$\text{SCORE} = \text{votos_netos} / (\text{horas_desde_publicacion} + 2)^{1.8}$$

Usada por el endpoint `GET /api/posts/trending` para devolver los 10 posts más relevantes del momento.

Índices creados

Índice	Tabla · Columna(s)	Beneficio
<code>idx_posts_game_id</code>	posts.game_id	Filtrado de posts por juego
<code>idx_posts_category</code>	posts.category	Filtrado por categoría
<code>idx_posts_author_id</code>	posts.author_id	Posts por usuario (perfil)
<code>idx_posts_created_at</code>	posts.created_at DESC	Ordenación "nuevos primero"
<code>idx_posts_published</code>	posts(is_published, is_deleted)	Filtro de visibilidad
<code>idx_comments_post_id</code>	comments.post_id	Carga de comentarios de un post
<code>idx_comments_parent</code>	comments.parent_id	Árbol de respuestas
<code>idx_votes_target</code>	votes(target_type, target_id)	Conteo de votos por post/comentario
<code>idx_likes_target</code>	likes(target_type, target_id)	Conteo de likes
<code>idx_revoked_tokens_exp</code>	revoked_tokens.expires_at	Limpieza de tokens caducados

5 API y endpoints

Base URL: `http://localhost:3000/api`. Formato: JSON. Autenticación: `Authorization: Bearer <accessToken>`.

Respuesta de error estándar:

```
{ "error": { "code": "UNAUTHORIZED", "message": "Token inválido o expirado" } }
```

Auth — /api/auth

Método	Ruta	Descripción	Auth
POST	<code>/api/auth/register</code>	Registro de nuevo usuario. Devuelve accessToken + cookie refreshToken	Pública
POST	<code>/api/auth/login</code>	Login con email+password. Devuelve accessToken + cookie refreshToken	Pública
POST	<code>/api/auth/refresh</code>	Renueva accessToken usando la cookie refreshToken (rotación automática)	Pública
POST	<code>/api/auth/logout</code>	Revoca el refreshToken y limpia la cookie	JWT

Posts — /api/posts

Método	Ruta	Descripción	Auth
GET	<code>/api/posts</code>	Lista paginada de posts. Params: <code>game</code> , <code>category</code> , <code>sort</code> (new/top/trending), <code>page</code> , <code>limit</code>	Pública
GET	<code>/api/posts/trending</code>	Top 10 posts por score de trending (vista post_scores)	Pública
GET	<code>/api/posts/search?q=</code>	Búsqueda full-text ILIKE en título y body	Pública
GET	<code>/api/posts/:id</code>	Post completo con media, steps, conteo de votos y likes	Pública
POST	<code>/api/posts</code>	Crear post con título, body, categoría, juego, media y steps opcionales	Usuario+
PUT	<code>/api/posts/:id</code>	Editar post propio (o cualquiera si rol es mod+)	Autor/Mod
DELETE	<code>/api/posts/:id</code>	Soft-delete del post (is_deleted = true)	Autor/Mod

Comentarios — /api/posts/:id/comments · /api/comments

Método	Ruta	Descripción	Auth
GET	/api/posts/:id/comments	Árbol anidado de comentarios. Deleted = [eliminado]	Pública
POST	/api/posts/:id/comments	Añadir comentario raíz o respuesta (parent_id opcional)	Usuario+
PUT	/api/comments/:id	Editar propio (solo el autor)	Autor
DELETE	/api/comments/:id	Soft-delete del comentario	Autor/Mod

Interacciones — votes · likes · favorites

Método	Ruta	Descripción
POST	/api/votes	Votar (upsert): { target_type, target_id, value: 1 -1 }
DELETE	/api/votes	Quitar voto: { target_type, target_id }
POST	/api/likes	Dar like. Devuelve { liked: true, like_count }
DELETE	/api/likes	Quitar like. Devuelve { liked: false, like_count }
POST	/api/favorites/:postId	Guardar post en favoritos
DELETE	/api/favorites/:postId	Eliminar de favoritos

Juegos — /api/games

Método	Ruta	Descripción	Auth
GET	/api/games	Lista de juegos con post_count y portada	Pública
GET	/api/games/:slug	Detalle de juego + últimos posts	Pública
GET	/api/games/search?q=	Búsqueda en IGDB con Apicalypse (debounce 350 ms en cliente)	Pública
POST	/api/games/search	Upsert de juego IGDB en la DB local (ON CONFLICT slug DO UPDATE)	Usuario+
POST	/api/games	Crear juego manualmente	Admin
PUT	/api/games/:slug	Editar juego	Admin

Usuarios — /api/users

Método	Ruta	Descripción	Auth
GET	/api/users/:username	Perfil público con followers_count, following_count, is_following	Pública
GET	/api/users/me	Mi perfil completo	JWT
PUT	/api/users/me	Actualizar username / bio / avatar_url	JWT
GET	/api/users/me/favorites	Mis favoritos paginados	JWT
GET	/api/users/search?q=	Búsqueda ILIKE de usuarios por username (para AddFriendModal)	JWT
GET	/api/users/:username/follow	Estado de seguimiento del usuario autenticado hacia el target	JWT
POST	/api/users/:username/follow	Seguir usuario (INSERT ON CONFLICT DO NOTHING)	JWT
DELETE	/api/users/:username/follow	Dejar de seguir usuario	JWT

Mensajería — /api/messages

Método	Ruta	Descripción	Auth
GET	/api/messages	Lista de conversaciones activas con último mensaje, timestamp y unread count (LATERAL join)	JWT
GET	/api/messages/:username	Mensajes con un usuario. Acepta <code>?since=ISO</code> para polling incremental. Marca mensajes recibidos como leídos.	JWT
POST	/api/messages/:username	Enviar mensaje (máx 2000 caracteres). Bloquea auto-mensajes.	JWT
GET	/api/messages/unread	Número de mensajes no leídos (para badge de Navbar, poll 10 s)	JWT

Amigos — /api/friends

Método	Ruta	Descripción	Auth
GET	/api/friends	Lista de amigos aceptados + solicitudes entrantes pendientes	JWT
POST	/api/friends/request	Enviar solicitud. Auto-acepta si ya existe solicitud inversa pendiente.	JWT
POST	/api/friends/respond	Aceptar o rechazar solicitud por <code>request_id</code>	JWT
GET	/api/friends/pending	Número de solicitudes entrantes pendientes (badge Navbar, poll 15 s)	JWT

Upload · Reportes · Admin · Health

Método	Ruta	Descripción	Auth
GET POST	/api/uploadthing	File router Uploadthing: <code>avatarUploader</code> (2 MB) y <code>postImageUploader</code> (8 MB). Auth vía JWT en middleware.	JWT
POST	/api/reports	Reportar post / comentario / usuario	Usuario+
GET	/api/admin/reports	Ver reportes (filtro por status)	Mod+
PUT	/api/admin/reports/:id	Resolver / desestimar reporte	Mod+
PUT	/api/admin/users/:id/ban	Banear / desbanear usuario	Mod+
DELETE	/api/admin/posts/:id	Eliminar cualquier post	Mod+
GET	/api/health	Health check: <code>{ status: "ok", db: "connected" }</code>	Pública

6 Sistema de autenticación

Arquitectura de tokens

Token	Duración	Almacenamiento	Uso
Access Token (JWT)	15 minutos	Memoria del cliente (React state)	Header <code>Authorization: Bearer</code> ... en cada request
Refresh Token (JWT)	7 días	Cookie <code>httpOnly</code> + <code>Secure</code> + <code>SameSite=Strict</code>	Renovar el access token sin relogin

Flujo de registro y login



Flujo de refresco de token (rotación)



Flujo de logout



Proxy (antes middleware) — src/proxy.ts

El archivo `src/proxy.ts` intercepta **todas las requests** a la aplicación. Si la request incluye un `Authorization: Bearer <token>` válido, extrae el payload del JWT y añade tres headers al request antes de pasarlo al route handler:

```
// Headers inyectados por el proxy en cada request autenticada
x-user-id      → payload.sub      // UUID del usuario
x-user-role    → payload.role     // admin | moderator | user | guest
x-user-username → payload.username
```

Cada route handler lee estos headers con `request.headers.get('x-user-id')`. Si el token es inválido o no existe, los headers no se añaden y la route handler devuelve 401 si requiere autenticación.

Matriz de permisos por rol

Acción	Invitado	Usuario	Moderador	Admin
Leer posts y comentarios	✓	✓	✓	✓
Votar / likear / favoritos	✗	✓	✓	✓
Crear post	✗	✓	✓	✓
Editar / eliminar post propio	✗	✓	✓	✓
Eliminar cualquier post	✗	✗	✓	✓
Gestionar reportes	✗	✗	✓	✓
Banear usuarios	✗	✗	✓	✓
Crear / editar juegos	✗	✗	✗	✓
Gestionar roles de usuario	✗	✗	✗	✓

Seguridad adicional implementada

- **Rate limiting:** login 10 req/15 min, registro 5 req/h, posts 10 req/h, votos 100 req/h — todo por IP o userId.
- **Sanitización XSS:** posts con DOMPurify permisivo (permite headings, listas, imágenes); comentarios con configuración restrictiva (solo formato básico).
- **Whitelist de embeds:** solo youtube.com,youtu.be y vimeo.com permitidos como `video_embed`.
- **SQL injection:** todas las queries usan `$1, $2, ...` (parametrizadas), nunca interpolación de strings.
- **Security headers:** CSP, X-Frame-Options: DENY, X-Content-Type-Options, HSTS, Referrer-Policy — configurados en `next.config.ts`.

A lo largo del desarrollo se detectaron y corrigieron varios problemas técnicos significativos.

Corrección 1 — `images.domains` → `images.remotePatterns`

¿Qué era el problema?

La propiedad `images.domains` de `next.config.ts` fue deprecada en Next.js 13 y eliminada en Next.js 16. Permitía indicar dominios permitidos para la optimización de imágenes, pero sin control sobre protocolo ni rutas.

¿Qué se cambió?

	Código antiguo (deprecado)	Código nuevo (correcto)
Propiedad	<code>images.domains:</code> <code>['images.example.com',</code> <code>'i.imgur.com']</code>	<code>images.remotePatterns: [{ protocol:</code> <code>'https', hostname: '...' }]</code>

```
// Antes (deprecado)
images: {
  domains: ['images.example.com', 'i.imgur.com'],
}

// Después (correcto - next.config.ts)
images: {
  remotePatterns: [
    { protocol: 'https', hostname: 'images.example.com' },
    { protocol: 'https', hostname: 'i.imgur.com' },
    // TODO: Añadir dominio CDN real cuando se configure el storage
  ],
},
```

Beneficio adicional: `remotePatterns` es más seguro porque permite especificar protocolo, hostname, pathname y port, evitando que se sirvan imágenes desde rutas inesperadas del mismo dominio.

Corrección 2 — `middleware.ts` → `proxy.ts` + renombrar función

¿Qué era el problema?

En Next.js 16, el archivo convencional `src/middleware.ts` fue renombrado a `src/proxy.ts` y la función exportada debe llamarse `proxy` (no `middleware`). El servidor mostraba: *"The 'middleware' file convention is deprecated. Please use 'proxy' instead."* y luego al crear el archivo: *"Proxy is missing expected function export name"*.

¿Qué se cambió?

	Antes	Después
Nombre del archivo	<code>src/middleware.ts</code>	<code>src/proxy.ts</code>
Nombre de la función exportada	<code>export async function middleware(request)</code>	<code>export async function proxy(request)</code>
Resto del código	Idéntico — misma lógica de inyección de headers JWT	

✓ Tras ambas correcciones, el servidor arrancó sin ningún aviso y el health check respondió correctamente:

```
{ "status": "ok", "db": "connected" }
```

Corrección 3 — isomorphic-dompurify crasheaba todas las API routes en Vercel

¿Qué era el problema?

Todas las rutas API que usaban `sanitize.ts` (posts, comentarios, usuarios) fallaban en producción con `ERR_REQUIRE_ESM`. La cadena de dependencias era: `isomorphic-dompurify → jsdom → html-encoding-sniffer → @exodus/bytes`. El paquete `@exodus/bytes` es ESM-only y Vercel/Turbopack no puede importarlo con `require()`.

¿Qué se cambió?

Se reemplazó completamente `isomorphic-dompurify` con una implementación nativa en `src/lib/sanitize.ts` que no tiene ninguna dependencia externa:

- `sanitizePlainText(text)` — elimina todas las etiquetas HTML y escapa entidades.
- `sanitizePostBody(html)` — whitelist de tags permitidos en posts (headings, listas, imágenes, enlaces, embeds).
- `sanitizeComment(html)` — whitelist restrictiva para comentarios (solo formato básico: `b, i, u, a, code`).

✓ Tras el fix, todos los endpoints de posts, comentarios y perfil volvieron a funcionar en producción.

Corrección 4 — Mensajes duplicándose por precisión de microsegundos

¿Qué era el problema?

El chat usaba `created_at` del último mensaje como parámetro `?since=` para el polling incremental. PostgreSQL almacena TIMESTAMPTZ con precisión de microsegundos (`.123456`), pero `JSON.stringify` trunca los timestamps de Date a milisegundos (`.123Z`). El mismo mensaje se devolvía en el siguiente poll porque `.123456 > .123000` pasaba el filtro `WHERE created_at > $since`.

¿Qué se cambió?

El cliente ahora suma 1 ms al timestamp antes de usarlo como `since`: `new Date(lastCreatedAt.getTime() + 1).toISOString()`. Además, se añadió deduplicación por `id` al añadir mensajes al estado para mayor robustez.

☒ Los mensajes ya no se duplican en el chat.

8

Estado actual y próximos pasos

¿Qué funciona hoy?

✓ Servidor corriendo en **http://localhost:3000** · Next.js 16.2.3 (Turbopack) · Ready in ~400ms

Capa	Estado	Detalles
Infraestructura	 Completo	PostgreSQL 17 corriendo, DB creada, 11 tablas, 10 índices
API de autenticación	 Completo	Register, login, refresh (rotación), logout (revocación)
API de posts	 Completo	CRUD, trending, búsqueda, paginación, filtros
API de comentarios	 Completo	Árbol anidado, create, edit, soft-delete
API de interacciones	 Completo	Votos (upsert), likes, favoritos
API de juegos	 Completo	CRUD por admin, detalle con últimos posts
API de usuarios	 Completo	Perfil público, mi perfil, edición, mis favoritos
API de moderación	 Completo	Reportes, baneos, eliminación de contenido
Seguridad	 Completo	Rate limiting, XSS sanitization, security headers, Zod validation
Frontend — componentes	 Completo	Navbar, Sidebar, Feed, PostCard, VoteButtons, CommentTree, GameSearch, FollowButton, AddFriendModal
Frontend — páginas	 Completo	19 páginas: home, post, game (IGDB), games, category, search, user, submit, settings, messages, messages/chat, friends, login, register, admin (×4)
Upload de imágenes	 Completo	Uploadthing v7: avatares en /settings, imágenes en formulario de post
IGDB	 Completo	Búsqueda en tiempo real, upsert en DB, páginas enriquecidas con screenshots y rating
Seguimiento	 Completo	Follow/unfollow, contadores en perfil, FollowButton con estado real desde cliente
Mensajería directa	 Completo	Chat con polling 3 s, marca de leído, badge en Navbar, lista de conversaciones
Sistema de amigos	 Completo	Solicitudes, aceptar/rechazar, lista de amigos, badge en Navbar

Qué falta por configurar

1. Búsqueda full-text optimizada (GIN)

La búsqueda actual usa ILIKE (suficiente para el volumen actual). Para producción con alto volumen de posts, migrar a índice GIN:

```
ALTER TABLE posts ADD COLUMN search_vector tsvector
  GENERATED ALWAYS AS (to_tsvector('spanish', title || ' ' || body)) STORED;
CREATE INDEX idx_posts_fts ON posts USING GIN(search_vector);
```

2. Rate limiter en Redis

El rate limiter actual usa memoria en proceso (sliding window). En Vercel serverless cada instancia tiene su propia memoria, por lo que el límite no es global. Para producción real migrar a Upstash Redis:

```
npm install @upstash/ratelimit @upstash/redis
```

Roadmap — próximas iteraciones

Prioridad	Feature	Notas
● Media	Migrar rate limiter a Upstash Redis	Necesario al escalar a múltiples instancias serverless
● Media	Índice GIN para búsqueda full-text	Mejora rendimiento con alto volumen de posts
● Media	Tests unitarios e integración	Cobertura mínima objetivo 70%
● Media	Notificaciones en tiempo real	WebSocket o SSE para reemplazar los pollings actuales
● Baja	Estado de amistad en perfiles	Mostrar "Amigos" / "Solicitud enviada" en /user/:username
● Baja	i18n (internacionalización)	Actualmente en español; arquitectura preparada para múltiples idiomas
● Baja	CDN para assets estáticos	Configurar en next.config.ts remotePatterns
● Baja	Moderación de mensajes privados	Reportar mensajes individuales

Credenciales del entorno de desarrollo

⚠ Las credenciales a continuación son solo para desarrollo local. No usar en producción.

Variable	Valor
PostgreSQL host	localhost:5432
PostgreSQL user	postgres
PostgreSQL database	forogaming
App URL	http://localhost:3000
JWT_ACCESS_EXPIRES	15m
JWT_REFRESH_EXPIRES	7d

Forogaming v1.2.0 · Documentación generada el 10 de abril de 2026

Proyecto desarrollado íntegramente con Claude Code (Anthropic) · Next.js 16 + PostgreSQL (Neon) + Uploadthing + IGDB

9 Despliegue en Producción

✅ Forogaming está desplegado y operativo en producción desde el 9 de abril de 2026. Stack: GitHub (código) + Vercel (hosting) + Neon (PostgreSQL serverless) — coste mensual: 0 €.

Infraestructura

Servicio	Proveedor	URL / Detalle	Plan
Frontend + API	Vercel	https://forogaming.vercel.app	Hobby (gratuito)
Base de datos	Neon	PostgreSQL 16 serverless · eu-west-2 (AWS Londres)	Free tier
Código fuente	GitHub	https://github.com/Nikode17/forogaming	Public repo
CI/CD	Vercel	Deploy automático en cada push a <code>master</code>	Incluido

Variables de entorno en producción (Vercel)

⚠️ Estos valores son privados. Se configuran en el panel de Vercel, nunca en el repositorio.

Variable	Descripción
<code>DATABASE_URL</code>	Connection string de Neon con <code>sslmode=require</code>
<code>JWT_SECRET</code>	Secreto de 48 bytes para firmar access/refresh tokens
<code>NEXT_PUBLIC_APP_URL</code>	<code>https://forogaming.vercel.app</code>
<code>APP_ENV</code>	<code>production</code> — activa SSL en el pool de PG y headers de seguridad

Proceso de despliegue paso a paso

1. **Instalación de gh CLI** — `winget install --id GitHub.cli`
2. **Autenticación GitHub** — `gh auth login` → Continue with GitHub (navegador)
3. **Inicialización del repositorio** — `git init`, commit inicial (69 ficheros), push
4. **Creación del repo en GitHub** — `gh repo create forogaming --public --source=. --push`
5. **Neon** — Proyecto *forogaming* creado manualmente en neon.tech; migraciones ejecutadas vía Node.js con `pg` Pool
6. **Vercel CLI** — `npm i -g vercel` · `vercel login` (GitHub OAuth)
7. **Variables de entorno** — `vercel env add DATABASE_URL production` (×4 variables)
8. **Deploy** — `vercel --prod --yes`; build exitoso en 38 s, 27 rutas generadas

Fix aplicado durante el despliegue

El build de producción falló por un error de TypeScript en `src/lib/db.ts`: el tipo genérico `T` de la función `query<T>` no tenía la restricción requerida por la librería `pg`.

```
// Antes (fallaba en producción)
export async function query<T = Record<string, unknown>>(...)
```



```
// Después (correcto)
export async function query<T extends QueryResultRow = QueryResultRow>(...)
```

El fix se commiteó, se hizo push y Vercel desplegó automáticamente en el siguiente push.

Panel de administración

Se añadieron dos mejoras al panel de admin tras el despliegue inicial:

- **Catálogo de juegos** (`/admin/games`) — CRUD completo: crear juego con slug automático, editar nombre/descripción/portada, listar con conteo de posts. Acceso restringido a rol `admin`.
- **Enlace en la Navbar** — El dropdown del usuario muestra "Panel de administración" en color indigo cuando `user.role === 'admin'`, enlazando a `/admin`.

Ruta	Descripción	Acceso
<code>/admin</code>	Dashboard con métricas y accesos rápidos	Admin
<code>/admin/games</code>	Gestión del catálogo de juegos (CRUD)	Admin
<code>/admin/users</code>	Gestión de usuarios (ban/unban)	Admin
<code>/admin/reports</code>	Revisión de reportes de contenido	Admin

Cuenta de administrador

El usuario **Nikode17** tiene rol `admin` en la base de datos de producción (Neon). El rol se asignó directamente vía SQL tras el registro en la web de producción:

```
UPDATE users SET role = 'admin' WHERE username = 'Nikode17';
```

Flujo CI/CD activo

Cada `git push` a la rama `master` desencadena automáticamente un nuevo build y deploy en Vercel. El tiempo medio de deploy es ~60 s. No se requiere ninguna acción manual.

Usuarios administradores

Usuario	Rol
Nikode17	admin
MuckMaster	admin

Variables de entorno adicionales (Uploadthing + IGDB)

Variable	Descripción
<code>UPLOADTHING_TOKEN</code>	Token JWT de la app Uploadthing (panel uploadthing.com)
<code>TWITCH_CLIENT_ID</code>	Client ID de la app Twitch para IGDB OAuth
<code>TWITCH_CLIENT_SECRET</code>	Client Secret de la app Twitch para IGDB OAuth

Forogaming v1.2.0 — en producción

<https://forogaming.vercel.app> · GitHub: Nikode17/forogaming

Actualizado el 10 de abril de 2026